

IB Computer Science

26 March 2026

Criterion C: Development

- 1) Structs for Mesh and Model
- 2) Scene and Node Traversal with Assimp
- 3) Texture Loading with STB Image
- 4) GPU Buffer setup with VAOs, VBOs, and EBOs
- 5) Shader Compilation and the Render Loop
- 6) Camera System and Inputs
- 7) Lighting System and ImGui layout

Criterion C: Development

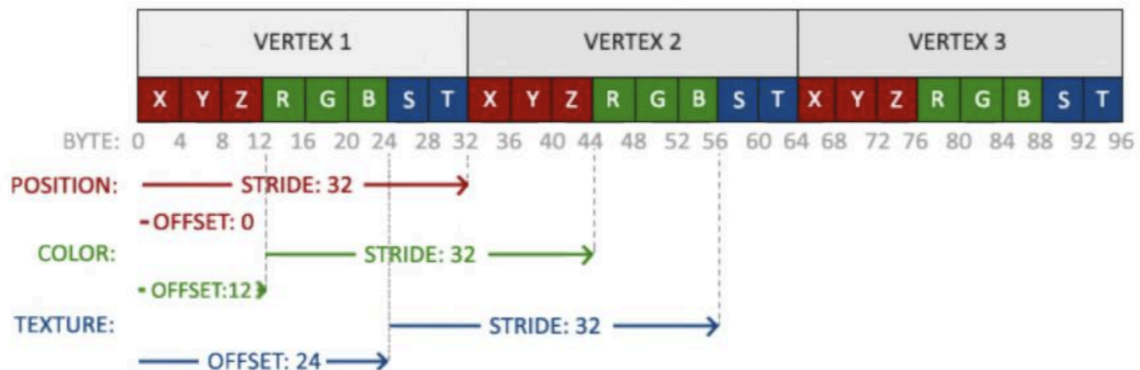
1. To represent the 3D data in memory, I needed a way to relate GPU objects together. I defined a Vertex struct with: position, normal, and texture coordinate data, and a Mesh struct. The VAO, VBO, EBO, index count, and texture into one unit. A Model then holds a vector of meshes. This made it easy to pass entire models between functions and kept the loading and drawing code clean.
2. Assimp loads a file into a scene graph of nodes, where each node can reference meshes and have child nodes. I wrote a recursive processNode function that walks this tree, calling buildMesh, and on each mesh it finds and adds the results to the output Model. One problem I ran into was that Assimp's string type caused a bug where the first four characters of texture file

names were being cut off. I fixed this by using `C_Str()` directly on the `aiString` rather than converting it to a standard string first.

3. I initially planned to use Assimp's built-in texture loading, but I found that it does not support flipping images on load, this causes textures to render upside down in OpenGL due to a difference in coordinate system difference. I switched to STB Image, which has a `stbi_set_flip_vertically_on_load` method that solves this. I also found that OBJ files/models already store textures in OpenGL's coordinate system, so I added a format detection method by reading the file extension and only flipping it for non-OBJ formats. This is to avoid loading the same texture multiple times.

4. Once the vertex and index data are generated using Assimp, it must be transferred to the GPU. A Vertex Array Object (VAO) was employed to encapsulate the attribute layout, while a Vertex Buffer Object (VBO) was utilized for vertex data, and an Element Buffer Object (EBO) was used for indices. The vertex attributes, specifically position, normal, and texture coordinates, are bound to locations 0, 1, and 2 via `glVertexAttribPointer`, thereby aligning with the layout anticipated by the vertex shader. The use of `GL_STATIC_DRAW` informs OpenGL that the data

will remain immutable, thereby enabling the driver to optimize data storage on the GPU.



5. Shaders are loaded from separate .vert and .frag files at runtime using fstream methods. This made editing the shaders so much faster since you didn't have to recompile the whole program to update them. The main render loop runs every frame, clearing the framebuffer, updating the view and projection matrices from the camera, and calling drawModel, which loops over meshes and calls glDrawElements. The loop also supports wireframe, solid with wireframe overlay, and point cloud modes just by changing OpenGL's polygon mode.

6. The camera is a struct storing: position, yaw, pitch, speed, and FOV values. Its front(), right(), and view() methods determine direction vectors and the view matrix using the GLM library. Dragging your mouse changes the yaw and pitch to rotate the camera's view, and the pitch is clamped to avoid flipping doing barrel rolls. Scroll moves the camera forward and backwards along its view direction, and WASD moves it relative to its own axes each frame like in most games, scaled by delta time so that the speed doesn't change depending on the frame rate..

7. Lighting is stored in a `LightingSettings` struct that supports up to four directional lights along with ambient, specular, emissive, rim light, fog, brightness, and contrast settings. These are sent to the shader through uniforms. The ImGui panel has buttons and settings for all of these controls and includes presets such as Studio, Sunset, Night, etc... that write preset values into the struct. I thought adding an FPS counter would be a good idea, and it works by sampling frame time every second and displays in the corner of the window.